CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 4 – CO4

Learning: Algorithm Design, Implementation, Test, Debug and Evaluate

Student Details	
Name	Abinekavarshini .M
Register Number	1924260333
Course Code	CSA17
Assignment	Assignment 4 – CO4
Total Marks	50

Prepared as a structured reference answer. Read, run and verify the code, and make sure you can explain each design decision before submission.

Important: The assignment requires actual code execution, plots and measured outputs. This document gives reproducible implementation guidance and hand calculations; do not report invented experimental metrics. Run the programs in your environment and insert your own output screenshots/tables.

How to use this answer

The assignment combines design and implementation. For each task, first understand the problem, state assumptions, write pseudocode and perform the manual trace. Then run the implementation, record the actual output and explain any difference. The code below is a starting implementation and may need small dataset/path adjustments depending on the Python and scikit-learn versions installed.

Common experimental practice: use a fixed random seed, split data before model selection, keep the test set untouched until final evaluation, and report precision, recall, F1-score and confusion matrix in addition to accuracy. For imbalanced classes, also report class-wise results.

Common notation

Entropy: H(S) = – Σc p(c) log■ p(c). Information gain: IG(S,A) = H(S) – Σv (|Sv|/|S|) H(Sv). A leaf predicts the majority class in its subset. In all code examples, random_state=42 is used for reproducibility.

TASK 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 Marks)

1.1 Diagnose the protocol

Deficiency	Why it is misleading
Training and testing on the same data	The model is evaluated on examples it has already seen; memorisation can look like generalisation.
No held-out test set	There is no independent final estimate of performance on unseen data.
No validation / cross-validation	Hyperparameters cannot be selected fairly; choices may be arbitrary or overfit.
Accuracy is the only metric	With an imbalanced at-risk class, predicting the majority class can give high accuracy while missing at-risk students.
No baseline	There is no simple reference, such as always predicting the majority class, to show whether the model adds value.
“Suitable model” is ambiguous	The algorithm, parameters, preprocessing and selection criterion are not specified, so the method is not reproducible.
No random seed / split protocol	Different runs may use different splits, making comparisons unstable and difficult to reproduce.
No leakage prevention	Preprocessing or feature selection performed before splitting can expose test information to training.

1.2 Repaired pseudocode

```
ALGORITHM EvaluateStudentRisk(D)
INPUT: labelled dataset D; candidate models M; fold count k
OUTPUT: selected model, final test metrics

1. Remove duplicates; define features X and target y.
2. Stratify-split D into development set Ddev (80%) and test set Dtest (20%).
3. Compute majority-class baseline from Ddev only.
4. FOR each candidate model m in M:
      FOR each hyperparameter setting h of m:
          Perform stratified k-fold cross-validation on Ddev.
          In each fold, fit preprocessing on fold-training data only.
          Train m(h); predict the fold-validation data.
          Store mean validation F1 and recall for the at-risk class.
5. Select the setting with the highest mean validation F1
   (use at-risk recall as a tie-breaker).
6. Refit the selected pipeline on all Ddev.
7. Evaluate it ONCE on Dtest.
8. Report accuracy, precision, recall, F1, confusion matrix,
   and majority-baseline metrics.
9. Return the fitted pipeline and test report.
END
```

The test set is not used for selecting the model or its hyperparameters. A stratified split preserves the approximate class proportions.

1.3 Demonstration code

```
# pip install scikit-learn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

model = DecisionTreeClassifier(random_state=42)
model.fit(Xtr, ytr)

train_acc = accuracy_score(ytr, model.predict(Xtr))
cv = cross_val_score(model, Xtr, ytr,
                     cv=StratifiedKFold(5, shuffle=True, random_state=42),
                     scoring="accuracy")
test_acc = accuracy_score(yte, model.predict(Xte))

print("Training accuracy:", train_acc)
print("CV mean accuracy:", cv.mean())
print("Held-out test accuracy:", test_acc)
print("Optimistic gap (train - CV):", train_acc - cv.mean())
```

Expected interpretation: a fully grown decision tree can fit its training examples extremely closely, so training accuracy may exceed cross-validated and held-out accuracy. The exact numbers depend on the installed library version and split; copy the values printed by your own run rather than using guessed values.

TASK 2 – Decision Trees (9 Marks)

2.1 Model the requirement

Input: attendance (High/Low), assignment status (Done/Missed), and internal marks (Good/Poor). Output: Pass or Fail. Objective: predict the result of a student and provide an interpretable sequence of attribute tests. Hypothesis space: finite categorical decision trees using the three attributes. Constraints: small dataset, categorical values, no missing values in the supplied trace. At each internal node, test one attribute; at each leaf, output a class. Information required: class frequencies in the current subset and the frequency of each class within every attribute value.

2.2 Splitting hypothesis

Choose the unused attribute with the greatest information gain, IG(S,A) = H(S) – weighted child entropy; this selects the attribute that most reduces class uncertainty at the current node and often produces a compact, informative tree.

2.3 Recursive tree construction pseudocode

```
ALGORITHM BuildTree(S, Attributes, ParentMajority)
1. IF S is empty: RETURN Leaf(ParentMajority).
2. IF all examples in S have the same class c: RETURN Leaf(c).
3. IF Attributes is empty: RETURN Leaf(MajorityClass(S)).
4. FOR each attribute A in Attributes:
      Compute H(S).
      For every value v of A, form Sv.
      Compute IG(S,A) = H(S) - Σv (|Sv|/|S|)H(Sv).
5. Let Best be the attribute with maximum information gain.
   Resolve ties using a fixed order: Attendance, Assignments, Internal Marks.
6. Create a decision node testing Best.
7. FOR each value v of Best:
      Sv = examples in S where Best=v.
      Child = BuildTree(Sv, Attributes - {Best}, MajorityClass(S)).
      Attach Child under branch v.
8. RETURN the decision node.
END
```

2.4 Desk-check: entropy and information gain

There are 8 examples: 4 Pass and 4 Fail. Therefore H(S) = –(4/8 log■ 4/8 + 4/8 log■ 4/8) = 1.000 bit.

Attribute	Weighted child entropy	Information gain
Attendance	0.8113	0.1887
Assignments	0.8113	0.1887
Internal Marks	0.8113	0.1887

All three attributes tie at the root. Using the stated fixed tie-break order, choose Attendance. For Attendance=High, the subset has 3 Pass and 1 Fail; entropy = 0.8113. For Attendance=Low, it has 1 Pass and 3 Fail; entropy = 0.8113.

Within either branch, splitting on Assignments gives information gain 0.3113; Internal Marks also gives 0.3113. The fixed tie-break chooses Assignments.

Branch	Assignments=Done	Assignments=Missed
Attendance=High	2 Pass, 0 Fail → Pass	1 Pass, 1 Fail → split on Internal Marks
Attendance=Low	1 Pass, 1 Fail → split on Internal Marks	0 Pass, 2 Fail → Fail

The two mixed subsets are resolved by Internal Marks: Good → Pass and Poor → Fail. The resulting tree is: Attendance? High: Assignments? Done→Pass, Missed: Internal Marks? Good→Pass, Poor→Fail. Low: Assignments? Done: Internal Marks? Good→Pass, Poor→Fail; Missed→Fail.

2.5 Implement and verify

```
# Compact categorical ID3 learner (information gain)
from collections import Counter
import math

def entropy(labels):
    n = len(labels)
    if n == 0: return 0.0
    counts = Counter(labels)
    return -sum((c/n)*math.log2(c/n) for c in counts.values())

def information_gain(rows, attr, target="Result"):
    parent = entropy([r[target] for r in rows])
    values = set(r[attr] for r in rows)
    child = sum(
        sum(r[attr] == v for r in rows)/len(rows) *
        entropy([r[target] for r in rows if r[attr] == v])
        for v in values)
    return parent - child

def majority(rows, target="Result"):
    return Counter(r[target] for r in rows).most_common(1)[0][0]

def build_tree(rows, attrs, target="Result", default=None):
    if not rows: return ("leaf", default)
    labels = [r[target] for r in rows]
    if len(set(labels)) == 1: return ("leaf", labels[0])
    if not attrs: return ("leaf", majority(rows, target))
    gains = [(information_gain(rows,a,target), a) for a in attrs]
    best = sorted(gains, key=lambda x:(-x[0], attrs.index(x[1])))[0][1]
    node = ("node", best, {})
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        node[2][v] = build_tree(subset, [a for a in attrs if a != best],
                                target, majority(rows,target))
    return node

def predict(tree, row):
    if tree[0] == "leaf": return tree[1]
    attr, branches = tree[1], tree[2]
    return predict(branches.get(row[attr], ("leaf", "Fail")), row)
```

For the required public-dataset comparison, use the same train/test split for your from-scratch learner and scikit-learn DecisionTreeClassifier. Report accuracy, precision, recall, F1 and confusion matrix using sklearn.metrics. A numeric comparison is valid only after running both implementations on the same data and split.

2.6 Overfitting and pruning

Train a fully grown tree, then compare it with a constrained tree such as max_depth=3 or min_samples_leaf=5. Record depth, number of leaves, training score, validation score and final test score. A full tree usually has lower training error but may have worse validation/test performance. Pruning reduces complexity and can improve generalisation, though excessive pruning causes underfitting.

2.7 Challenge

Change one label, for example change example 4 from Fail to Pass. Recompute all entropies and gains before deciding whether the root changes. A unique identifier has many distinct values and can create an apparently high information gain by isolating individual records; this is memorisation, not a useful predictive split. Repair: remove identifier fields, use gain ratio or minimum leaf-size constraints, and validate on held-out data.

TASK 3 – Hidden Variables and the EM Algorithm (6 Marks)

3.1 Exact defect

The responsibility r[i][k] is calculated as an unnormalised joint score $\pi[k] \times \text{Normal}(x[i] \mid \mu[k], \text{var}[k])$. It is not yet a posterior probability. The missing operation is normalisation across all K components for each data point. Consequently, $\Sigma k\ r[i][k]$ is generally not 1, so Nk and the parameter updates do not have their intended expected-count meaning.

3.2 Failing input and trace

Choose X = [–2, –1, 1, 2], K=2, π=[0.5,0.5], μ=[–1,1], variances=[1,1]. For x=0, both Gaussian densities are equal: $\phi(0|-1,1)=\phi(0|1,1)\approx 0.2420$. The faulty scores are [0.1210, 0.1210], whose sum is 0.2420, not 1. The correct responsibilities are [0.5,0.5]. This is the first incorrect value: the responsibility row.

3.3 Minimal repair

```
# Replace the responsibility assignment with:
scores = [pi[k] * normal_pdf(X[i], mu[k], var[k]) for k in range(K)]
denominator = sum(scores)
if denominator == 0:
    raise ValueError("All mixture likelihoods are zero; use stable log-space code.")
for k in range(K):
    r[i][k] = scores[k] / denominator
```

For numerical stability in production, compute log π[k] + log density, subtract the maximum log score, exponentiate and normalise.

3.4 Implementation: corrected EM core

```
import numpy as np

def gaussian_pdf(x, mu, var):
    return np.exp(-0.5*(x-mu)**2/var) / np.sqrt(2*np.pi*var)

def em_1d(X, K=2, iterations=100, seed=42, faulty=False):
    rng = np.random.default_rng(seed)
    X = np.asarray(X, dtype=float)
    pi = np.ones(K)/K
    mu = rng.choice(X, K, replace=False).astype(float)
    var = np.full(K, max(np.var(X), 1e-3))
    loglikes, weight_sums = [], []
    for _ in range(iterations):
        scores = np.column_stack([
            pi[k]*gaussian_pdf(X, mu[k], var[k]) for k in range(K)])
        if faulty:
            r = scores
        else:
            r = scores / scores.sum(axis=1, keepdims=True)
        Nk = r.sum(axis=0)
        if np.any(Nk <= 1e-12):
            raise ValueError("Empty component; reinitialise it.")
        mu = (r*X[:,None]).sum(axis=0)/Nk
        var = (r*(X[:,None]-mu)**2).sum(axis=0)/Nk
        var = np.maximum(var, 1e-6)
        pi = Nk/len(X)
        # Evaluate observed-data log likelihood with updated parameters
        mix = sum(pi[k]*gaussian_pdf(X,mu[k],var[k]) for k in range(K))
        loglikes.append(np.log(np.maximum(mix,1e-300)).sum())
        weight_sums.append(pi.sum())
    return pi, mu, var, np.array(loglikes), np.array(weight_sums)
```

Run both modes on your chosen data, plot loglikes and weight_sums, and repeat with at least 20 seeds. The repaired EM should have non-decreasing observed-data log-likelihood up to floating-point tolerance when each E-step and M-step is implemented correctly. The faulty version is not guaranteed to have this property. Do not claim a result until your plots and 20-run experiment have been executed.

3.5 Regression check

Use a one-component mixture (K=1) as a regression case. The correct responsibility is exactly 1 for every data point; the faulty score is the Gaussian density and is generally not 1. This reveals that the original responsibility logic is not mathematically correct even in the simplest case. If the original implementation had a separate normalisation elsewhere, test that path too.

TASK 4 – Neural Network Back-Propagation (9 Marks)

4.1 Define the state and dimensions

For a fully connected network, let layer l have n_l units. With batch size m: W^(l) has shape (n_l × n_(l–1)); b^(l) has shape (n_l × 1); Z^(l)=W^(l)A^(l–1)+b^(l) and A^(l)=g_l(Z^(l)). For one example, vectors z^(l), a^(l), δ^(l) each have n_l entries. δ^(l)=∂Loss/∂z^(l), the derivative of loss with respect to pre-activation values.

4.2 Forward and backward recurrences

Forward: a^(0)=x; z^(l)=W^(l)a^(l–1)+b^(l); a^(l)=g_l(z^(l)). For squared error L=½||a^(L)–y||², output boundary: δ^(L)=(a^(L)–y) ■ g'_L(z^(L)). Hidden recurrence: δ^(l)=((W^(l+1))■δ^(l+1)) ■ g'_l(z^(l)). Gradients: ∂L/∂W^(l)=δ^(l)(a^(l–1))■ and ∂L/∂b^(l)=δ^(l). Errors are computed backwards because the chain rule for a layer depends on the error at the next layer.

4.3 Mini-batch training pseudocode

```
ALGORITHM TrainNetwork(D, sizes, η, batchSize, maxEpochs)
1. Initialise each W with small random values; initialise b to zero.
2. Repeat for epoch = 1..maxEpochs:
      Shuffle training examples.
      Divide examples into mini-batches.
      For each batch B:
          Set accumulated gradients to zero.
          For each (x,y) in B:
              Compute all z and a values (forward pass).
              Compute output δ from the loss derivative.
              Compute hidden δ values from last layer to first.
              Accumulate δ(l)(a(l-1))■ and δ(l).
          Divide accumulated gradients by |B|.
          Update W(l) ← W(l) – η gradW(l).
          Update b(l) ← b(l) – η gradb(l).
      Evaluate validation loss.
      Stop if validation loss has not improved for patience epochs
      or maxEpochs is reached.
3. Return parameters from the epoch with best validation loss.
END
```

4.4 Desk-check: 2-2-1 sigmoid network

Assume each listed pair is the incoming weight vector for one hidden neuron, inputs are x=(1,0), all biases are zero, output weights are (0.6,–0.2), target y=1, sigmoid σ(t)=1/(1+e^(–t)), and loss L=½(y–o)².

Quantity	Calculation	Value
Hidden 1	σ(0.5×1 + (–0.4)×0)	0.622459
Hidden 2	σ(0.3×1 + 0.8×0)	0.574443
Output net	0.6(0.622459) – 0.2(0.574443)	0.258587
Output	σ(0.258587)	0.564289
Loss	½(1–0.564289)²	0.094922

Output delta δo=(o–y)o(1–o)=–0.107127. Hidden deltas: δh1=0.6δo·h1(1–h1)=–0.015105; δh2=–0.2δo·h2(1–h2)=0.005238. With η=0.5, output weights become (0.633341, –0.169231). Hidden weights become neuron 1: (0.507553, –0.4); neuron 2: (0.297381, 0.8). Bias updates: output +0.053563; hidden 1 +0.007553; hidden 2 –0.002619.

4.5 NumPy implementation and checks

```
import numpy as np
sigmoid = lambda z: 1/(1+np.exp(-z))

def forward(x, W1, b1, W2, b2):
    h = sigmoid(W1 @ x + b1)
    o = sigmoid(W2 @ h + b2)
    return h, o

def loss_and_grads(x, y, W1, b1, W2, b2):
    h, o = forward(x,W1,b1,W2,b2)
    L = 0.5*np.sum((o-y)**2)
    do = (o-y)*o*(1-o)
```

```
    dW2 = do @ h.T
    db2 = do
    dh = (W2.T @ do)*h*(1-h)
    dW1 = dh @ x.T
    db1 = dh
    return L,dW1,db1,dW2,db2

# Numerical gradient check for each parameter:
# For parameter θj, grad_num ≈ [L(θj+ε)-L(θj-ε)]/(2ε).
# Relative error = ||g_num-g_analytic|| /
#                  max(1e-12, ||g_num||+||g_analytic||).
```

For XOR, a single-layer perceptron cannot represent the XOR truth table because XOR is not linearly separable. A network with a hidden layer can represent XOR; train it and report predictions for (0,0), (0,1), (1,0), (1,1).

4.6 Stress experiment

Use sklearn.datasets.load_digits (8×8 images). Standardise pixel values using training data only. Test hidden units {2,8,32,64}, learning rates {0.001,0.01,0.1,1.0}, and activations {sigmoid, ReLU}. Record loss curves, accuracy, macro-F1 and confusion matrix for each setting. A very large learning rate can cause unstable/diverging loss; sigmoid networks with small initial weights can show vanishing gradients. Record the norm of each layer's gradient to diagnose where gradients become very small or unusually large.

TASK 5 – Same Dataset, Different Learning Reasoning (8 Marks)

5.1 Naive Bayes with Laplace smoothing

For class c, estimate P(c)=N_c/N. For categorical feature j and value v, estimate P(x_j=v|c)=(N_{jvc}+α)/(N_c+αV_j), where V_j is the number of possible values. Classify by the class with maximum log score log P(c)+Σj log P(x_j|c). Laplace smoothing (α=1) prevents zero likelihoods.

Toy zero-count example: suppose all three examples are Normal and none is Hazardous with feature value “High temperature”. With α=0, P(High temperature|Hazardous)=0, so the product likelihood for a hazardous test record becomes zero. With α=1 and two possible temperature values, the estimate is (0+1)/(0+2)=0.5, avoiding a zero probability.

Naive Bayes pseudocode

```
EstimateNB(D, α):
  Count class frequencies N_c.
  For each class c and feature j and value v:
      P(c) = N_c / N
      P(v|c) = (N_jvc + α) / (N_c + α V_j)
  Return probability tables.

PredictNB(x):
  For each class c:
      score[c] = log P(c) + Σj log P(xj|c)
  Return class with greatest score.
```

Implementation experiment: use a labelled text or tabular dataset, split once with stratification, and evaluate α∈{0,0.1,1,10}. α=0 may require a small numerical floor for log(0), but report that it represents unsmoothed maximum-likelihood estimation. Record accuracy, precision, recall, F1 and log-loss.

5.2 Kernel machines

The unsafe assumption is that a linear decision boundary is sufficient to separate hazardous and normal states. A linear SVM chooses a maximum-margin hyperplane. Support vectors are training points closest to the boundary and determine its position. A kernel computes inner products in an implicit higher-dimensional feature space. The RBF kernel K(x,z)=exp(−γ||x−z||²) can model nonlinear boundaries.

Compare SVC(kernel='linear') and SVC(kernel='rbf') using GridSearchCV with 5-fold stratified cross-validation over C and, for RBF, γ. Report support-vector count, accuracy, F1 and fit time. Compare Naive Bayes, your Task 2 tree and both SVMs on the identical held-out split. Plot decision boundaries only when using two selected features; otherwise use dimensionality reduction for visualisation and state that it is only a projection.

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
cv = StratifiedKFold(5, shuffle=True, random_state=42)
grid = GridSearchCV(
```

```
    make_pipeline(StandardScaler(), SVC()),
    {"svc__kernel":["linear","rbf"],
     "svc__C":[0.1,1,10,100],
     "svc__gamma":["scale",0.01,0.1,1]},
    scoring="f1", cv=cv, n_jobs=-1)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
```

5.3 Explanation-Based Learning

A possible domain theory (first-order style) for an overload warning is:

```
R1: HighCurrent(s) AND NormalVoltage(s) -> CurrentStress(s).
R2: HighCurrent(s) AND LowVoltage(s) -> ElectricalOverload(s).
R3: ElectricalOverload(s) -> Hazardous(s).
R4: CurrentStress(s) AND HighTemperature(s) -> ThermalStress(s).
R5: ThermalStress(s) -> Hazardous(s).
R6: NormalCurrent(s) AND NormalTemperature(s) -> NormalState(s).
R7: NormalState(s) AND NormalVoltage(s) -> Safe(s).
R8: Hazardous(s) -> IssueAlert(s).
```

Example explanation: a case with HighCurrent and LowVoltage proves ElectricalOverload by R2; R3 then proves Hazardous; R8 derives IssueAlert. Generalising the proof gives the operational rule HighCurrent(s) AND LowVoltage(s) → IssueAlert(s). The learned rule is a compact operational form of a conclusion already entailed by the domain theory; it does not add new facts to the theory. Test it on 20 designed cases and count inference steps for direct rule application versus full proof.

TASK 6 – Reinforcement Learning for an Energy-Aware Smart Controller (13 Marks)

Stage 1 – Interpret the problem

Input/observation: occupancy estimate, indoor temperature or comfort band, outdoor temperature, current time slot/electricity price, and recent controller state. Output/action: Off, Low or High cooling. Objective: minimise electricity cost while maintaining comfort when occupied. Constraints: discrete 15-minute decisions, three available actions, comfort requirements during occupancy, equipment limits and uncertain future occupancy/weather/price. An episode can represent one operating day, starting at the first time slot and ending after the final slot.

Stage 2 – Strategy selection

A fixed thermostat is a useful baseline but cannot adapt to changing electricity prices. Supervised imitation requires labelled operator actions and may copy inefficient behaviour. Q-learning is suitable when the transition model is unknown and the action/state space is discretised; it learns action values from interaction. SARSA is on-policy and may be more conservative under its exploration policy. A model-based method requires a reliable transition model. Proposed choice: tabular Q-learning for a small discretised simulator, with SARSA as an optional comparison.

Stage 3 – State, action and reward

State s_t = (occupancy band, indoor-temperature band, outdoor-temperature band, price band, time-of-day band, previous action). This is approximately Markov if it contains the variables needed to predict the next state; if temperature dynamics depend on longer history, add recent temperature trend or use a richer state. Actions A={Off, Low, High}. Define reward $r_t = -(\text{price}_t \times \text{energy_kWh}_t) - \lambda \times \text{comfort_violation}_t$, where comfort_violation_t is positive when occupied and temperature is outside the allowed comfort band. Choose λ large enough that the learner cannot cheaply avoid energy cost by making occupied periods uncomfortable. Use ε-greedy exploration and discount factor γ, for example 0.95.

Stage 4 – Q-learning pseudocode

```
ALGORITHM TrainSmartAC(episodes, α, γ, ε)
1. Initialise Q(s,a)=0 for every state-action pair.
2. FOR episode = 1..episodes:
      Initialise environment at start of a day; observe state s.
      WHILE episode has not ended:
          With probability ε choose a random action a.
          Otherwise choose a with maximum Q(s,a).
          Execute a for 15 minutes.
          Observe reward r and next state s'.
          IF s' is terminal:
              target = r
          ELSE:
```

```
            target = r + γ max_b Q(s',b)
          Q(s,a) ← Q(s,a) + α[target − Q(s,a)].
          s ← s'.
      Reduce ε according to the exploration schedule.
3. Extract policy π(s)=argmax_a Q(s,a).
4. Evaluate π on separate days/seeds against fixed thermostat baseline.
5. RETURN Q and π.
END
```

Stage 5 – Four-update desk-check

Define three states: S1=occupied and warm, S2=occupied and comfortable, S3=unoccupied. Two actions: A1=Low and A2=Off. Let α=0.5 and γ=0.9; initialise Q=0. For transparent arithmetic, assume the following observed transitions and rewards.

Step	Transition	Reward	Updated Q value
1	S1 —Low→ S2	–1	Q(S1,Low)=0.5[–1+0–0]=–0.50
2	S2 —Low→ S2	–0.2	Q(S2,Low)=0.5[–0.2+0–0]=–0.10
3	S2 —Off→ S3	–0.5	Q(S2,Off)=0.5[–0.5+0–0]=–0.25
4	S3 —Off→ terminal	0	Q(S3,Off)=0.5[0–0]=0.00

This is a deliberately small illustrative trace. For a nonterminal update with nonzero next-state Q values, include γ max_b Q(s',b) in the target. The full simulator should use physically plausible temperature dynamics and energy measurements.

Stage 6 – Implementation plan and adversarial test

Build a simulator with a temperature update equation, for example T_in(t+1)=T_in(t)+heat_gain(outdoor, occupancy)–cooling_effect(action), with noise and bounds. Estimate energy for Off/Low/High, multiply by the current tariff, and calculate comfort violation only when occupied. Run at least 20 random seeds. Report mean and standard deviation of episode reward, energy (kWh), comfort violations and episodes to convergence for Q-learning and the fixed-rule baseline.

Adversarial test: create a schedule where occupants arrive shortly after a high-price period. A reward that penalises only current energy cost may encourage Off just before arrival, leaving the room too warm during occupancy. Predict this failure before running. If observed, add a pre-cooling/arrival forecast feature to the state and/or a stronger occupied-comfort penalty, then rerun the experiment. Report both the original and repaired results.

```
# Minimal Q-learning update used by the simulator
import numpy as np
Q = np.zeros((n_states, n_actions))
alpha, gamma, epsilon = 0.1, 0.95, 0.2

def update_q(s, a, reward, next_s, done):
    future = 0.0 if done else np.max(Q[next_s])
    target = reward + gamma * future
    Q[s, a] += alpha * (target - Q[s, a])
```

Final submission checklist

• Include problem interpretation, inputs, outputs, objective and constraints for every task.

• Show design pseudocode before implementation code.

• Include manual traces and intermediate calculations, not only final answers.

• Run every program and attach actual outputs, plots and metrics.

• Use a fixed random seed and keep the held-out test data separate from model selection.

• Do not invent accuracy, F1, likelihood, convergence or timing values.

• Explain one failure case and the correction made for each required challenge.

• Check that the final PDF includes your own verified results and follows the SIMATS pseudocode handbook.

Reference note: The task structure, data tables, required evidence and marking expectations in this answer follow the uploaded CSA17 Assignment 4 document. The numerical desk-check values are calculated from the assignment's supplied examples; experimental metrics must be produced by running the included code.

Clarification – Task 3.5 Regression Check

A valid regression input on which the original unnormalised responsibility calculation happens to be correct is a one-component mixture with X=[0], K=1, $\pi_1=1$, $\mu_1=0$ and variance $\sigma_1^2=1/(2\pi)$. At x=0, the Gaussian density is $1/\sqrt{(2\pi\sigma^2)}=1$. Therefore the original score is $\pi_1\times$density=1, and the repaired responsibility is 1/1=1. Both versions produce the same responsibility and therefore preserve the correct result for this deliberately chosen case.

This is a special numerical case, not evidence that the original formula is generally correct. For ordinary data and parameters, the unnormalised scores do not sum to one. The corrected implementation must normalise responsibilities across components for every data point.